

LAB Assignment No 6

Interfacing LED & Potentiometer with ESP32 and Visualizing Data on Node-RED Dashboard

LAB Description:

This lab demonstrates how IoT devices communicate with cloud-based dashboards using MQTT, enabling real-time monitoring and control.

Learning Objectives:

By the end of this lab, students will:

- Understand IoT communication using MQTT
- Interface sensors (potentiometer) with ESP32
- Control actuators (LED) remotely
- Build a web dashboard using Node-RED
- Simulate IoT systems using Wokwi

Required Tools

🔧 Hardware (Simulated)

- ESP32
- LED
- Potentiometer

💻 Software

- Arduino IDE
- Node-RED
- MQTT Broker (e.g., HiveMQ or Mosquitto)

System Overview

Potentiometer → ESP32 → MQTT → Node-RED Dashboard (Display)

Node-RED Button → MQTT → ESP32 → LED Control

Potentiometer:

- VCC → 3.3V

- GND → GND
- OUT → GPIO 36

LED:

- Positive → GPIO 26
- Negative → GND

ESP32 Tasks

Students must:

1. Connect ESP32 to WiFi
2. Read potentiometer value
3. Publish data to MQTT topic:

pubtopic

4. Subscribe to MQTT topic:

LED

5. Turn LED ON/OFF based on received message

 Node-RED Tasks

Students must create a dashboard:

 MQTT Configuration

- Broker: `broker.hivemq.com`
- Port: `1883`

◆ Flow Requirements

1. Display Sensor Data

- MQTT IN → Topic:

pubtopic

- Connect to:
 - UI Text / Gauge (Potentiometer value)

2. LED Control

- Add two buttons:

Button	Payload	Topic
ON	ON	LED
OFF	OFF	LED

- Connect to MQTT OUT
-

ESP32 code

```
#include <WiFi.h>
#include <PubSubClient.h>

// ===== WiFi =====
const char* ssid = "Wokwi-GUEST";
const char* password = "";

// ===== MQTT =====
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;

// ===== Pins =====
#define ADCpin 36
#define LED 26

int ValueToSend;

// ===== Objects =====
WiFiClient espClient;
PubSubClient client(espClient);

// ===== CALLBACK =====
void callback(char* topic, byte* payload, unsigned int length) {
```

```

Serial.println();
Serial.print("Message received on topic: ");
Serial.println(topic);

String msg = "";

for (int i = 0; i < length; i++) {
    msg += (char)payload[i];
}

Serial.print("Message: ");
Serial.println(msg);

if (String(topic) == "LED") {
    if (msg == "ON") {
        digitalWrite(LED, HIGH);
        Serial.println("LED ON");
    }
    else if (msg == "OFF") {
        digitalWrite(LED, LOW);
        Serial.println("LED OFF");
    }
}
}

// ===== WIFI CONNECT =====
void connectWiFi() {
    Serial.print("Connecting to WiFi");
    WiFi.begin(ssid, password);

    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    Serial.println("\nWiFi Connected!");
    Serial.print("IP Address: ");
    Serial.println(WiFi.localIP());
}

// ===== MQTT RECONNECT =====
void reconnectMQTT() {
    while (!client.connected()) {
        Serial.println("Connecting to MQTT...");

        String clientId = "ESP32Client-";

```

```

clientId += String(random(1000));

// PUBLIC BROKER → NO USERNAME/PASSWORD
if (client.connect(clientId.c_str())) {
    Serial.println("MQTT Connected!");

    client.subscribe("LED"); // Subscribe topic
}
else {
    Serial.print("Failed, state = ");
    Serial.println(client.state());
    delay(2000);
}
}
}

// ===== SETUP =====
void setup() {
    Serial.begin(115200);

    pinMode(LED, OUTPUT);

    connectWiFi();

    client.setServer(mqttServer, mqttPort);
    client.setCallback(callback);
}

// ===== LOOP =====
void loop() {

    if (!client.connected()) {
        reconnectMQTT();
    }

    client.loop();

    // Read ADC value
    int ADC = analogRead(ADCpin);
    ValueToSend = map(ADC, 0, 4095, 0, 100);

    Serial.print("Pot Value (%): ");
    Serial.println(ValueToSend);

    // Convert to string
    char payload[10];

```

```
snprintf(payload, sizeof(payload), "%d", ValueToSend);

// Publish to MQTT
client.publish("pubtopic", payload);

delay(2000);
}
```

Dashboard URL

Students must verify:

```
http://127.0.0.1:1880/ui
```

(or `/dashboard` if using newer dashboard)

Expected Output

- ✓ Rotating potentiometer → value updates on dashboard
- ✓ Clicking ON → LED turns ON in Wokwi
- ✓ Clicking OFF → LED turns OFF

Student Deliverables

1. Screenshot of:
 - Wokwi simulation
 - Node-RED flow
 - Dashboard UI
2. Code submission:
 - ESP32 code
 - Node-RED flow (export JSON)
3. Short explanation:
 - How MQTT works in this system

Viva Questions:

- What is MQTT and why is it used in IoT?
- Difference between publish and subscribe?
- Why do topics need to match exactly?
- What happens if broker is down?
- What is the role of Node-RED?

Bonus Tasks (Optional)

- Add slider to control LED brightness (PWM)
- Add real-time graph for potentiometer
- Add threshold alert (e.g., value > 80)

Docs

Simulation

00:19.394 101%

0

load:0x40080400,len:2972
entry 0x400805dc
Connecting to WiFi.....
WiFi Connected!
IP Address: 10.10.0.2
Connecting to MQTT...
MQTT Connected!



sketch.ino

diagram.json

libraries.txt

Library Manager



```
1  #include <WiFi.h>
2  #include <PubSubClient.h>
3
4  // WiFi
5  const char* ssid = "Wokwi-GUEST";
6  const char* password = "";
7
8  // MQTT
9  const char* mqttServer = "broker.hivemq.com";
10 const int mqttPort = 1883;
11
12 // Pins
13 #define ADCpin 35
14 #define LED 26
15
16 WiFiClient espClient;
17 PubSubClient client(espClient);
18
19 // ===== CALLBACK =====
20 void callback(char* topic, byte* payload, unsigned int length) {
21     String msg = "";
22
23     for (int i = 0; i < length; i++) {
24         msg += (char)payload[i];
25     }
```

```

26
27     Serial.print("Message: ");
28     Serial.println(msg);
29
30     if (String(topic) == "LED") {
31         if (msg == "ON") {
32             digitalWrite(LED, HIGH);
33         } else if (msg == "OFF") {
34             digitalWrite(LED, LOW);
35         }
36     }
37 }
38
39 // ===== WIFI =====
40 void connectWiFi() {
41     WiFi.begin(ssid, password);
42
43     while (WiFi.status() != WL_CONNECTED) {
44         delay(500);
45         Serial.print(".");
46     }
47
48     Serial.println("WiFi Connected!");
49 }

```

```

51 // ===== MQTT =====
52 void reconnectMQTT() {
53     while (!client.connected()) {
54         if (client.connect("ESP32Client")) {
55             client.subscribe("LED");
56         } else {
57             delay(2000);
58         }
59     }
60 }
61
62 // ===== SETUP =====
63 void setup() {
64     Serial.begin(115200);
65
66     pinMode(LED, OUTPUT);
67
68     connectWiFi();
69
70     client.setServer(mqttServer, mqttPort);
71     client.setCallback(callback);
72 }
73
74 // ===== LOOP =====
75 void loop() {

```

```

74 // ===== LOOP =====
75 void loop() {
76
77     if (!client.connected()) {
78         reconnectMQTT();
79     }
80
81     client.loop();
82
83     int ADC = analogRead(ADCpin);
84     int value = map(ADC, 0, 4095, 0, 100);
85
86     Serial.println(value);
87
88     char payload[10];
89     sprintf(payload, "%d", value);
90
91     client.publish("pubtopic", payload);
92
93     delay(2000);
94 }

```

Flow 1 Flow 2 Flow 3 + debug i 📄 🛠 ⚙ ▼

▼ all nodes 🗑 all

29/04/2026, 23:33:57 node: debug 2
msg.payload : string[2]
"ON"

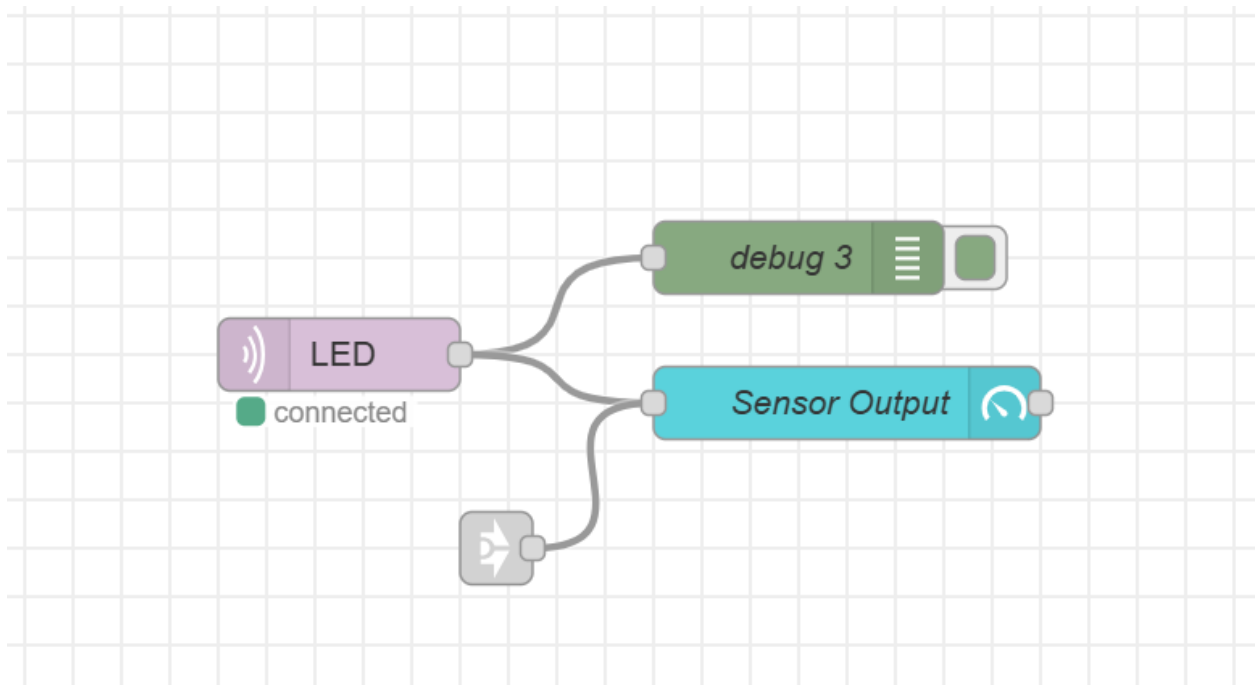
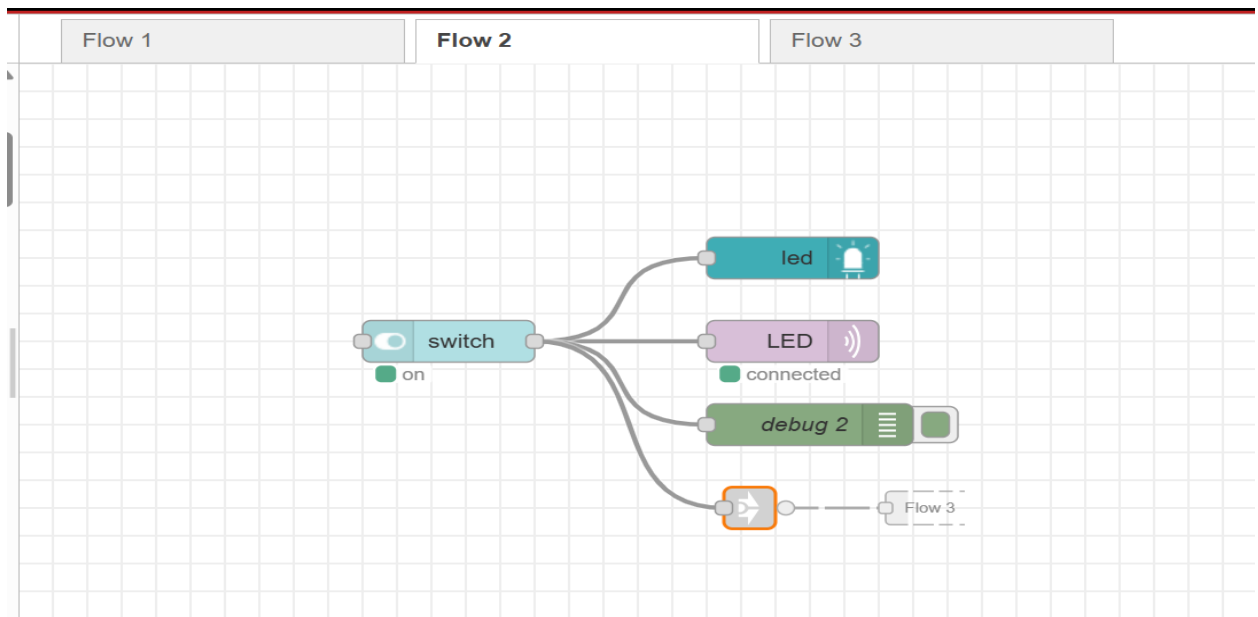
29/04/2026, 23:33:58 node: debug 3
LED : msg.payload : string[2]
"ON"

29/04/2026, 23:33:59 node: debug 2
msg.payload : string[3]
"OFF"

29/04/2026, 23:33:59 node: debug 3
LED : msg.payload : string[3]
"OFF"

29/04/2026, 23:34:05 node: debug 2
msg.payload : string[2]
"ON"

29/04/2026, 23:34:06 node: debug 3



Simulation

01:19.857 52%

58

68

71

70

70

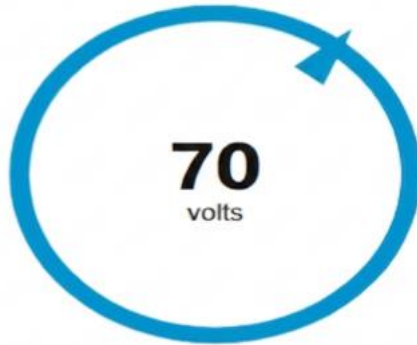
70

70

Sensor Output



gauge



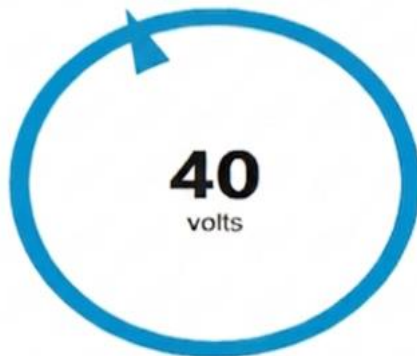
switch



Sensor Output



gauge



switch

